

Facultad de Negocios y Tecnologías

Licenciatura en Ingeniería de Software

SEDCM

INTEGRANTES

- ALEJANDRO BERMÚDEZ BÁEZ
- 2
- 3
- 4
- 5

PROFESOR: JORGE H.

.

Tabla de contenido

Tabla de contenido	2
Documentación Técnica: Smart Edge Data Center Manager (SEDCM)	3
1. Introducción y Resumen Ejecutivo	3
2. Definición del Alcance y Producto Mínimo Viable (MVP)	3
3. Variables de Lectura (Telemetría de Entrada)	4
4. Actuadores y Efectores (Control de Salida)	4
5. Requerimientos del Sistema	5
5.1 Requerimientos Funcionales (RF)	5
5.2 Requerimientos No Funcionales (RNF - Atributos de Calidad)	6
6. Diseño de Arquitectura y Distribución en Red (Despliegue LAN On-Premise)	7
6.1 Nodos de Borde (Edge Nodes en LAN)	7
6.2 Capa de Transporte y Enrutamiento (Middleware Local)	8
6.3 Nodo Central de Procesamiento (Control Plane)	8
7. Modelo de Datos y Estructura de la Información	8
7.1 Esquemas de Mensajería de Red (Payloads MQTT en JSON)	8
7.2 Esquema Relacional de Persistencia (PostgreSQL)	9
8.0 Reglas de Consumo Eléctrico (Sostenibilidad y Gestión de Recursos)	10
9.0 Reglas de Salud de Software (Disponibilidad y Tolerancia a Fallas)	11
10. Reglas de Temperatura (Integridad Física)	13
Diagramas	14
1.-Diagrama de componentes (MVP)	14
2.- Diagrama de componentes por zona	14

Documentación Técnica: Smart Edge Data Center Manager (SEDCM)

1. Introducción y Resumen Ejecutivo

En la actualidad, la gestión operativa de los Centros de Datos exige respuestas en milisegundos ante anomalías térmicas o picos de consumo de recursos. El proyecto **Smart Edge Data Center Manager (SEDCM)** es un sistema distribuido de grado industrial diseñado para el monitoreo y control automatizado de infraestructura crítica.

A diferencia de los sistemas de monitoreo pasivos tradicionales, SEDCM opera bajo el paradigma de **Edge Computing**. Al procesar las reglas de operación en el borde de la red (LAN), el sistema es capaz de tomar decisiones autónomas (Industria 4.0) para maximizar la eficiencia energética y proteger la integridad física del hardware mediante acciones preventivas, como la parada de emergencia de servicios. El sistema garantiza la tolerancia a fallos y la resiliencia al utilizar un middleware de mensajería asíncrona (MQTT), lo que permite gestionar activos de red de forma transparente a nivel de zonas, racks y servidores.

2. Definición del Alcance y Producto Mínimo Viable (MVP)

Para demostrar la viabilidad de la arquitectura distribuida y la comunicación IoT en red, el Producto Mínimo Viable (MVP) simulará la topología lógica y física de un Data Center utilizando tecnología de contenerización (Docker) y un modelo de red modular (LAN). El sistema operará bajo los siguientes parámetros:

- **Topología de Red Modular:** El despliegue contará con un equipo físico dedicado exclusivamente como "Servidor Central" (Control Plane). Para la infraestructura de borde, se conectará un mínimo de 2 equipos físicos adicionales, donde cada equipo representará lógicamente una "Zona" (ej. Zona A y Zona B). La arquitectura permite agregar \$N\$ cantidad de Zonas de forma transparente. Cada Zona administrará internamente la simulación de 2 Racks (ej. A1 y A2).
- **Gestión de Nodos (Cargas de Trabajo Simuladas):** En cada rack operarán contenedores Docker que actuarán como los servidores/nodos a monitorear. Para fines del MVP, estos contenedores no ejecutarán software de estrés real, sino scripts generadores de telemetría controlada. Estos scripts emularán comportamientos de carga (Normal, Advertencia y Crítico) para disparar las reglas del sistema de manera predecible durante la demostración.

- **Ingesta de Telemetría (Granularidad Híbrida):** La recolección de datos provendrá de simuladores estructurados en dos niveles:
 - **Métricas de Nodo (Por Contenedor):** Scripts internos en cada contenedor generarán datos emulados de Uso de CPU (%), Consumo de RAM (MB) e I/O de Red. El sistema evaluará estas métricas para determinar el *Estado de Salud* del nodo (ej. si satura la RAM, se considera en estado crítico).
 - **Métricas Ambientales (Por Rack):** Scripts a nivel de borde simularán variables físicas de temperatura (°C) y humedad relativa (%) independientes para cada rack. El motor de reglas cruzará estos datos con el estado de los actuadores de ventilación para ajustar la simulación térmica (lazo cerrado).

3. Variables de Lectura (Telemetría de Entrada)

El sistema recolectará datos periódicamente, los cuales serán generados por scripts de emulación diseñados para inyectar cargas de trabajo controladas y predecibles durante la demostración. Estos datos se dividen según su alcance lógico y físico:

Variable a Medir	Categoría	Granularidad (Alcance)	Unidad de Medida	Origen / Sensor
Uso de CPU	Métrica de Nodo	Por Contenedor	Porcentaje (%)	Script Emulador (Nodo)
Consumo de RAM	Métrica de Nodo	Por Contenedor	Megabytes (MB)	Script Emulador (Nodo)
I/O de Red	Métrica de Nodo	Por Contenedor	Tasa Tx/Rx (Bytes/s)	Script Emulador (Nodo)
Temperatura	Ambiental	Por Rack	Grados Celsius (°C)	Script Simulador (Borde)
Humedad Relativa	Ambiental	Por Rack	Porcentaje (%)	Script Simulador (Borde)

4. Actuadores y Efectores (Control de Salida)

Para mitigar riesgos operativos (saturación de recursos) y térmicos, el sistema cuenta con efectores de software y simuladores de hardware que responden a los comandos emitidos por el Motor de Reglas central a través del broker MQTT.

Componente Actuador	Tipo	Granularidad (Alcance)	Acción de Control (Comando)	Interfaz de Ejecución
Edge Executor	Software	Por Contenedor	soft_reboot (Reinicio preventivo)	API local de Docker
Edge Executor	Software	Por Contenedor	hard_shutdown (Apagado indefinido)	API local de Docker
Módulo HVAC	Físico (Simulado)	Por Rack	set_hvac_mode (cooling, humidify, dehumidify)	Cliente MQTT en el rack
Extractores	Físico (Simulado)	Por Rack	set_fan_speed (Ajustar RPM)	Cliente MQTT en el rack

5. Requerimientos del Sistema

5.1 Requerimientos Funcionales (RF)

- **RF-01: Registro y Gestión de Inventario Dinámico**
 - **Tipo:** Funcional | **Prioridad:** Alta
 - **Descripción:** El sistema debe registrar una jerarquía lógica de activos (Zonas y Racks). La identificación de los contenedores (nodos) debe ser dinámica, basada en las etiquetas (*labels*) de Docker (ej. dc.zone, dc.rack), almacenando y actualizando este estado en la base de datos PostgreSQL. *(Nota: Se elimina la dependencia de etiquetas de criticidad).*
- **RF-02: Recolección y Normalización de Telemetría Emulada**
 - **Tipo:** Funcional | **Prioridad:** Alta
 - **Descripción:** El componente *Edge Collector* debe capturar las métricas de rendimiento por nodo (CPU, RAM, Red I/O) directamente desde los **scripts emuladores internos**, empaquetarlas junto con un *timestamp* de origen en un objeto JSON estandarizado, y publicarlas en el broker MQTT bajo el tópico correspondiente a su ubicación.
- **RF-03: Simulación Ambiental y Actuadores Físicos**
 - **Tipo:** Funcional | **Prioridad:** Media
 - **Descripción:** El sistema en el borde debe incluir scripts que simulen la lectura por rack de temperatura y humedad relativa. Asimismo, debe proveer interfaces virtuales (tópicos MQTT) que simulen la recepción de comandos para los actuadores físicos de infraestructura (módulo HVAC para refrigeración y control de extractores).
- **RF-04: Motor de Reglas y Detección de Anomalías de Salud**
 - **Tipo:** Funcional | **Prioridad:** Alta
 - **Descripción:** El Backend debe procesar el flujo de telemetría proveniente de los scripts emuladores. Si las métricas de un nodo (CPU o RAM) superan los umbrales de seguridad (ej. >90%) por un tiempo sostenido, el sistema debe realizar una transición de estado del nodo de "Normal" a "Crítico". Este estado representa una **falla lógica simulada** que activa el protocolo de respuesta automática.

- **RF-05: Protocolo de Mitigación por Escalación (Ciclo de Vida de Falla)**
 - **Tipo:** Funcional | **Prioridad:** Alta
 - **Descripción:** Ante la detección de un nodo en estado "Crítico" (Falla), el sistema debe ejecutar una secuencia de comandos MQTT para intentar la recuperación. La primera acción será un soft_reboot (reinicio del contenedor); si tras un intervalo definido el script emulador sigue reportando métricas críticas, el sistema escalará la acción a un hard_shutdown (apagado indefinido) para proteger la estabilidad de la Zona.
- **RF-06: Persistencia Histórica y Bitácora de Auditoría**
 - **Tipo:** Funcional | **Prioridad:** Alta
 - **Descripción:** El nodo central debe garantizar que la telemetría se almacene como series de tiempo en PostgreSQL. Se debe registrar obligatoriamente una bitácora inmutable que rastree qué comando se emitió, el motivo del disparo, y si el nodo de borde confirmó su ejecución exitosa mediante un acuse de recibo (ACK).
- **RF-07: Visualización y Alertas en Tiempo Real (Dashboard)**
 - **Tipo:** Funcional | **Prioridad:** Alta
 - **Descripción:** El sistema debe proveer una interfaz gráfica accesible vía web en la red local. Esta interfaz mantendrá una conexión viva mediante WebSockets con el Backend para reflejar cambios de estado en la jerarquía, telemetría y alertas críticas con latencia casi nula, sin requerir recargas de página.
- **RF-08: Intervención Manual y Control de Recuperación Segura**
 - **Tipo:** Funcional | **Prioridad:** Media
 - **Descripción:** La interfaz gráfica debe permitir al operador emitir comandos manuales hacia los nodos de borde (ej. reiniciar un servicio o encender ventiladores manualmente tras un hard_shutdown). El sistema debe bloquear automáticamente las acciones de reinicio de software si los umbrales ambientales físicos no se encuentran dentro del rango de "Recuperación Segura" (ej. Temperatura $\leq 42^{\circ}\text{C}$).

5.2 Requerimientos No Funcionales (RNF - Atributos de Calidad)

RNF-01: Seguridad en el Transporte de Datos (LAN)

- **Atributo:** Seguridad | **Prioridad:** Alta
- **Descripción:** Aunque el sistema opere en un entorno de red local aislada (LAN), toda comunicación del middleware debe estar estrictamente cifrada mediante el protocolo MQTTS. Se utilizarán certificados TLS/SSL autogenerados para prevenir la interceptación (ataques *Man-in-the-Middle*) o la manipulación maliciosa de los comandos de control físicos.

RNF-02: Desempeño y Latencia Crítica

- **Atributo:** Desempeño | **Prioridad:** Alta
- **Descripción:** El sistema debe operar en tiempo casi real (*Near Real-Time*). El tiempo transcurrido desde que un sensor de borde publica una métrica crítica en el broker, hasta que el Motor de Reglas evalúa la anomalía y emite el comando de mitigación correspondiente, no debe exceder los 2 segundos de latencia de red end-to-end.

RNF-03: Resiliencia y Autocorrección en Nodos de Borde

- **Atributo:** Tolerancia a Fallas y Disponibilidad | **Prioridad:** Alta
- **Descripción:** Los agentes desplegados en el borde (Collector y Executor) deben soportar desconexiones intermitentes de la red física o caídas del servidor central. Ante la pérdida de conexión con el Broker MQTT, deben implementar un algoritmo de reintento con retroceso exponencial (*Exponential Backoff*) y reanudar sus suscripciones de forma autónoma una vez que la red se restablezca, sin requerir reinicios manuales.

RNF-04: Transparencia de Ubicación y Desacoplamiento de Red

- **Atributo:** Escalabilidad y Mantenibilidad | **Prioridad:** Alta
- **Descripción:** El *Backend* central no debe tener conocimiento absoluto de las direcciones IP físicas de los nodos de borde. Toda interacción se realizará obligatoriamente mediante el paradigma asíncrono de Publicación/Suscripción del broker. Esto garantizará que se puedan agregar nuevos racks, zonas o simuladores a la red en el futuro sin modificar ni recompilar el código base del servidor central.

RNF-05: Portabilidad y Despliegue Estandarizado

- **Atributo:** Portabilidad | **Prioridad:** Media
- **Descripción:** Para garantizar la reproducibilidad del entorno On-Premise, todos los componentes del sistema (Backend, Broker MQTT, Base de Datos y Nodos de Borde) deben estar contenerizados. El despliegue de la infraestructura lógica se realizará mediante archivos de configuración estandarizados (ej. docker-compose), asegurando la compatibilidad a través de diferentes sistemas operativos anfitriones.

RNF-06: Integridad de la Bitácora de Auditoría

- **Atributo:** Integridad de Datos | **Prioridad:** Alta
- **Descripción:** Los registros almacenados en la tabla de auditoría (comandos emitidos y ACKs recibidos) deben tratarse bajo el principio de inmutabilidad operativa. El sistema no proveerá interfaces en la API ni en el Dashboard para la modificación o eliminación de estos registros históricos, garantizando un rastro confiable para análisis forense tras incidentes térmicos o eléctricos.

6. Diseño de Arquitectura y Distribución en Red (Despliegue LAN On-Premise)

El sistema SEDCM se implementará como un sistema distribuido que opera sobre una Red de Área Local (LAN), utilizando un modelo de comunicación asíncrono basado en eventos (Arquitectura Orientada a Eventos). Físicamente, el sistema estará segmentado en múltiples equipos computacionales conectados al mismo conmutador (Switch) o punto de acceso inalámbrico, divididos en las siguientes capas lógicas:

6.1 Nodos de Borde (Edge Nodes en LAN)

Esta capa representa la topología modular física. Estará conformada por equipos físicos (ej. Laptops de los desarrolladores) donde cada computadora representará lógicamente una "Zona" independiente (Zona A, Zona B, etc.). Cada Zona administrará internamente la simulación de sus respectivos racks (ej. A1 y A2). En cada equipo operarán los siguientes componentes:

- **Nodos de Carga (Contenedores Docker):** Servidores virtuales monitoreados por el sistema. Ejecutarán internamente los scripts emuladores encargados de generar cargas predecibles de CPU, RAM e I/O de red para la demostración.

- **Simuladores de Hardware (Sensores y Actuadores):** Scripts locales a nivel de rack que emulan la lectura de variables ambientales (Temperatura y Humedad Relativa) y representan a los equipos físicos de mitigación (módulo HVAC y extractores).
- **Edge Collector (Agente Python):** Un servicio local que extrae periódicamente las métricas de software de los contenedores emulados y las métricas físicas de los simuladores ambientales. Transforma estos datos en payloads JSON y los publica en la red.
- **Edge Executor (Agente Python):** Un servicio residente que mantiene una suscripción a su tópico de control. Al recibir comandos de escalación, consume la API de Docker del host local para ejecutar reinicios preventivos (soft_reboot) o apagados (hard_shutdown), e interactúa con los simuladores para encender/apagar la refrigeración.

6.2 Capa de Transporte y Enrutamiento (Middleware Local)

Esta capa actúa como el bus de mensajes asíncrono de la red interna.

- **Broker MQTT (Eclipse Mosquitto):** Instalado en un equipo físico designado exclusivamente como Servidor Central (Control Plane). Para cumplir con la seguridad en la red, incluso siendo local, operará bajo el protocolo MQTTS utilizando certificados TLS/SSL autogenerados para cifrar los comandos de control. Este middleware garantiza el desacoplamiento total de los nodos y otorga tolerancia a las desconexiones temporales de los equipos de borde.

6.3 Nodo Central de Procesamiento (Control Plane)

Este componente reside en el servidor principal de la red local, centralizando la toma de decisiones y la persistencia.

- **Motor de Reglas y Backend (Node.js/TypeScript):** Aplicación que consume la telemetría del broker MQTT en la red, evalúa en tiempo real las reglas de operación para calcular el **"Estado de Salud"** de la infraestructura, y emite de forma autónoma los comandos de escalación cuando se detectan fallas simuladas.
- **Base de Datos (PostgreSQL):** Alojada localmente junto al backend, encargada de la persistencia relacional del estado actual, el inventario jerárquico y la bitácora de auditoría inmutable de los comandos ejecutados.
- **Dashboard (Interfaz de Visualización - React):** Se sirve desde el nodo central. Cualquier otro equipo en la red local (ej. la computadora del profesor evaluador) podrá ingresar mediante la dirección IP local del servidor y mantener una conexión

abierta bidireccional por WebSockets para ver gráficos, topología de red y alertas en tiempo real.

7. Modelo de Datos y Estructura de la Información

El sistema utilizará un modelo de datos unificado que permitirá representar de manera coherente tanto la información transmitida en tiempo real entre los componentes distribuidos como la información almacenada de forma persistente para análisis histórico, monitoreo y auditoría. Este enfoque permite que los datos mantengan una misma estructura lógica desde su generación en los emuladores y servicios edge, pasando por el broker MQTT y el backend, hasta su almacenamiento en PostgreSQL.

7.1 Esquemas de Mensajería de Red (Payloads MQTT en JSON)

La comunicación entre procesos distribuidos se realizará enviando objetos JSON a través del broker. Para respetar la granularidad de los datos y evitar la saturación de la red, se definen las siguientes estructuras principales:

A.1 Payload de Telemetría de Nodo (Publicado por Edge Collector)

Este mensaje contiene las métricas de software emuladas y se publicará periódicamente en el tópico: dc/telemetria/zona/{Z}/rack/{R}/nodo/{N}.

```
{
  "timestamp": "2026-04-06T13:15:00Z",
  "metadata": {
    "dc_zone": "A",
    "dc_rack": "A1",
    "node_id": "nodo_web_01"
  },
  "metrics": {
    "cpu_usage_pct": 98.5,
    "ram_usage_mb": 2048.0,
    "net_rx_bytes_sec": 5000,
    "net_tx_bytes_sec": 8500
  }
}
```

(Nota: Integramos los metadatos de ubicación directamente en el payload para que el motor de reglas no tenga que consultar a la base de datos de dónde viene el dato).

A.2 Payload de Telemetría Ambiental (Publicado por Edge Collector) Este mensaje representa el estado físico del rack y se publicará en un tópico menos granular: `dc/telemetria/zona/{Z}/rack/{R}/ambiente`.

```
{
  "timestamp": "2026-04-06T13:15:00Z",
  "metadata": {
    "dc_zone": "A",
    "dc_rack": "A1"
  },
  "environment": {
    "temperature_c": 38.5,
    "humidity_pct": 45.0
  }
}
```

B. Payload de Comando de Control (Publicado por el Backend)

Cuando el motor de reglas detecta una anomalía, emitirá un comando estructurado para que el *Edge Executor* actúe. El tópico de destino será `dc/control/zona/{Z}/rack/{R}`.

```
{
  "command_id": "cmd-98765",
  "timestamp_issued": "2026-04-06T13:16:02Z",
  "target": {
    "dc_zone": "A",
    "dc_rack": "A1",
    "target_type": "nodo",
    "target_id": "nodo_web_01"
  },
  "action": "soft_reboot",
  "reason": "Estado Crítico: CPU sostenido > 95%"
}
```

7.2 Esquema Relacional de Persistencia (PostgreSQL)

Para representar fielmente la arquitectura distribuida y separar las métricas físicas de las lógicas, la base de datos central se normalizará en un esquema de 6 tablas principales agrupadas en tres dominios:

Dominio de Inventario (Jerarquía Físico-Lógica)

1. **zonas:** Define las áreas geográficas modulares del Data Center (Ej. Zona A = Laptop 1).
2. **racks:** Representa la infraestructura física virtualizada. Contiene una llave foránea a zonas y su estado térmico actual.
3. **nodos (antes contenedores):** Representa las cargas de trabajo simuladas. Contiene una llave foránea a racks y un campo dinámico estado_salud (Valores: 'Normal', 'Warning', 'Critico') que es actualizado en tiempo real por el Motor de Reglas.

Dominio de Telemetría (Series de Tiempo Separadas)

4. **telemetria_nodo:** Almacena las métricas emuladas de software de cada nodo (CPU, RAM, Red Tx/Rx). Se vincula mediante llave foránea a la tabla nodos.
5. **telemetria_ambiental:** Almacena las variables físicas simuladas (Temperatura y Humedad). Se vincula directamente a la tabla racks, evitando la duplicación masiva de datos ambientales por nodo.

Dominio de control.

6. **bitacora_auditoria:** Registro inmutable de los comandos generados por el motor de reglas (Ej: soft_reboot, hard_shutdown o set_cooling). Almacena la orden, el motivo de la falla y el timestamp de la confirmación (ACK) recibida desde el nodo de borde, cerrando el lazo de control.

8.0 Reglas de Consumo de Recursos (Escalación de Incidentes)

El consumo de energía y la estabilidad de un nodo son directamente proporcionales a la utilización de su CPU y memoria RAM. Para garantizar la disponibilidad del rack, el motor de reglas evaluará continuamente la telemetría emulada y aplicará las siguientes políticas de escalación automática:

1. **Estado Normal:** Uso de CPU y RAM por debajo del 80%. Operación estándar.
2. **Estado Warning (Saturación Temprana):** Si el uso de CPU o RAM de un nodo supera el 80% durante más de 3 ciclos de lectura consecutivos.
 - **Acción:** El sistema transiciona el estado del nodo a "Warning" y emite una alerta naranja al Dashboard. No se toman acciones destructivas, pero se registra en la bitácora.
3. **Estado Crítico (Peligro de Colapso - OOM/Overheating):** Si las métricas emuladas del nodo superan el 95% de uso.
 - **Acción Automática:** El Motor de Reglas inicia el protocolo de escalación. Primero, emite un comando al *Edge Executor* para aplicar un reinicio preventivo (soft_reboot). Si tras un periodo de gracia las métricas continúan en estado crítico, escala la acción emitiendo un apagado indefinido (hard_shutdown).
 - **Objetivo:** Contener el fallo lógico y evitar que un nodo descontrolado afecte el rendimiento general de su Zona.
4. **Estado de Recuperación:** Cuando el consumo global del rack desciende y se estabiliza, el Dashboard habilita la opción para que el administrador envíe un comando manual de encendido/reinicio a los nodos apagados.

9. Reglas de Salud de Software (Disponibilidad y Tolerancia a Fallas)

En sistemas distribuidos, debemos asumir que los fallos de red ocurrirán. Estas reglas garantizan que el sistema detecte anomalías lógicas o de conectividad antes de que colapsen la demostración.

1. Regla de Silencio de Telemetría (Timeout / Heartbeat Loss):

- **Condición:** Si el Backend deja de recibir mensajes MQTT de un nodo o rack específico por más de 30 segundos.
- **Acción:** Se marca el activo con estado OFFLINE en la base de datos. Se dispara una alarma crítica de "Pérdida de Comunicación", asumiendo una desconexión de la Zona (laptop física apagada o desconectada de la LAN).

2. Regla de Anomalía de I/O de Red (Detección de Bloqueos):

- **Condición:** Si el CPU está al 100% pero las métricas de red (Tx/Rx) caen a 0 Bytes/sec (indicador de un posible *Deadlock* o bloqueo de hilo del script emulador).
- **Acción:** Notificación de "Anomalía de Servicio". El sistema ejecuta un `soft_reboot` preventivo de manera autónoma para intentar romper el bloqueo lógico.

3. Regla de Resiliencia del Agente (Autocorrección en el Borde):

- **Condición:** Si el servicio *Edge Collector* pierde conexión con el Broker MQTT central.
- **Acción:** Intervención autónoma local. El agente debe reintentar la conexión con un retroceso exponencial (*Exponential Backoff*) sin intervención manual de los desarrolladores, restableciendo el flujo en cuanto la red LAN vuelva a estar disponible.

10. Reglas Ambientales (Integridad Física Simulada)

El hardware físico es susceptible tanto al sobrecalentamiento como a las anomalías en el aire. Las desviaciones en las métricas ambientales simuladas dispararán acciones sobre los actuadores virtuales del rack (HVAC).

10.1 Reglas de temperatura

Regla	Condición	Acción Técnica	Justificación
Optimo	Temp <=27°C	Notificar estado óptimo.	Temperatura en rango seguro de operación.
Warning	temp >= 28°C & <=44°C	Disparar comando set_hvac_mode (Modo: Cooling).	Precaución; se enciende el aire acondicionado simulado para contrarrestar el calentamiento.
Critical	temp >= 45°C	Disparar comando hard_shutdown a nodos del rack.	El enfriamiento falló. Apagado para evitar daños permanentes al procesador.

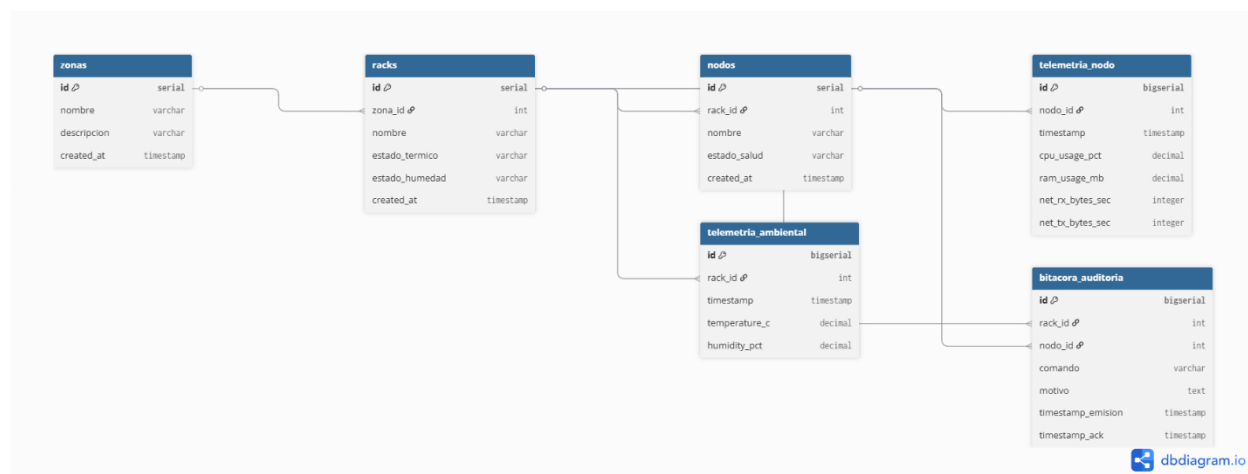
10.2 Reglas de humedad relativa

Regla	Condición	Acción Técnica	Justificación
Óptimo	Humedad entre 40% y 60%	Notificar estado óptimo.	Rango ideal para evitar estática y condensación.
Warning (Seco)	Humedad < 40%	Disparar comando set_hvac_mode (Modo: Humidify).	El aire está muy seco. Riesgo de acumulación de electricidad estática en los componentes.
Warning (Húmedo)	Humedad > 60%	Disparar comando set_hvac_mode (Modo: Dehumidify).	Hay demasiada humedad. Riesgo de condensación y cortocircuitos en las placas base.
Critical	Humedad < 20% o > 80%	Disparar comando hard_shutdown a nodos del rack.	Ambiente físicamente hostil. Apagado preventivo inmediato ante riesgo inminente de cortocircuito o

			descarga estática severa.
--	--	--	------------------------------

11. Diagramas

11.1 Entidad – Relacion



11.1.1 Diccionario de Datos

Para que no tengas que escribir todo a mano en Word, aquí tienes la estructura base de tu Diccionario de Datos. Puedes copiar estas tablas a tu documento. Representan el dominio de Control, que es el más importante para la calificación:

Tabla: bitacora_auditoria

(Registra de forma inmutable todas las acciones de mitigación emitidas por el sistema).

Campo	Tipo de Dato	Llave	Nulo	Descripción
id	BigSerial	PK	NO	Identificador único del registro de auditoría.
rack_id	Integer	FK	NO	Relación con el Rack afectado.
nodo_id	Integer	FK	SI	Relación con el Nodo afectado (Nulo si el comando fue para hardware físico como HVAC).
comando	Varchar(50)	-	NO	Acción ejecutada (soft_reboot, hard_shutdown, set_cooling).
motivo	Text	-	NO	Justificación emitida por el motor de reglas (Ej. "Temp > 45°C").

timestamp_emision	Timestamp	-	NO	Sello de tiempo exacto en que el Backend publicó el JSON en MQTT.
timestamp_ack	Timestamp	-	SI	Sello de tiempo en que el <i>Edge Executor</i> confirmó la ejecución.

Tabla: nodos

(Mapea los contenedores emulados y su estado dinámico calculado).

Campo	Tipo de Dato	Llave	Nulo	Descripción
id	Serial	PK	NO	Identificador único del nodo en el sistema central.
rack_id	Integer	FK	NO	Relación física con el gabinete donde se aloja.
nombre	Varchar(100)	-	NO	Nombre del contenedor en Docker (ej. nodo_web_01).
estado_salud	Varchar(20)	-	NO	Estado calculado en tiempo real (Normal, Warning, Critico).
created_at	Timestamp	-	NO	Fecha de registro del nodo en el inventario.

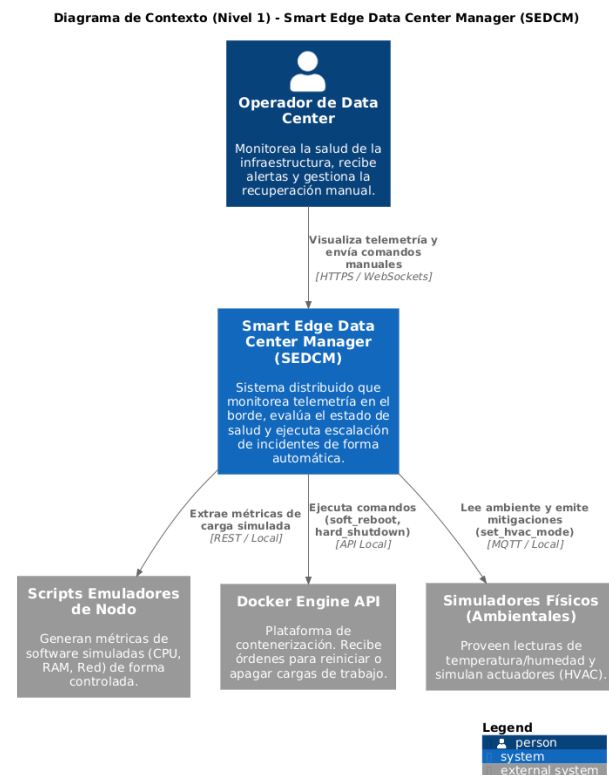
(Nota: Puedes crear tablas similares a estas para telemetria_nodo y telemetria_ambiental usando la información del diagrama de DBML).

11.2 Diagramas de arquitectura C4

11.2.1 C1 Diagrama de contexto

El modelo C4, el Nivel 1 se centra en el valor del negocio y las fronteras del sistema. La base de datos y el middleware son decisiones técnicas de implementación que nos pertenecen; por lo tanto, viven dentro de la caja azul del SEDCM. Los actores y sistemas externos son:

1. **El Usuario:** El Operador o Administrador que vigila el *Dashboard*.
2. **Nuestro Sistema:** SEDCM.
3. **Dependencia Externa 1:** Scripts Emuladores de Nodo (Generan las métricas de software simuladas).
4. **Dependencia Externa 2:** La API de Docker (ejecuta comandos de mitigación como reinicios o apagados).
5. **Dependencia Externa 3:** Los simuladores de sensores y actuadores físicos del rack.



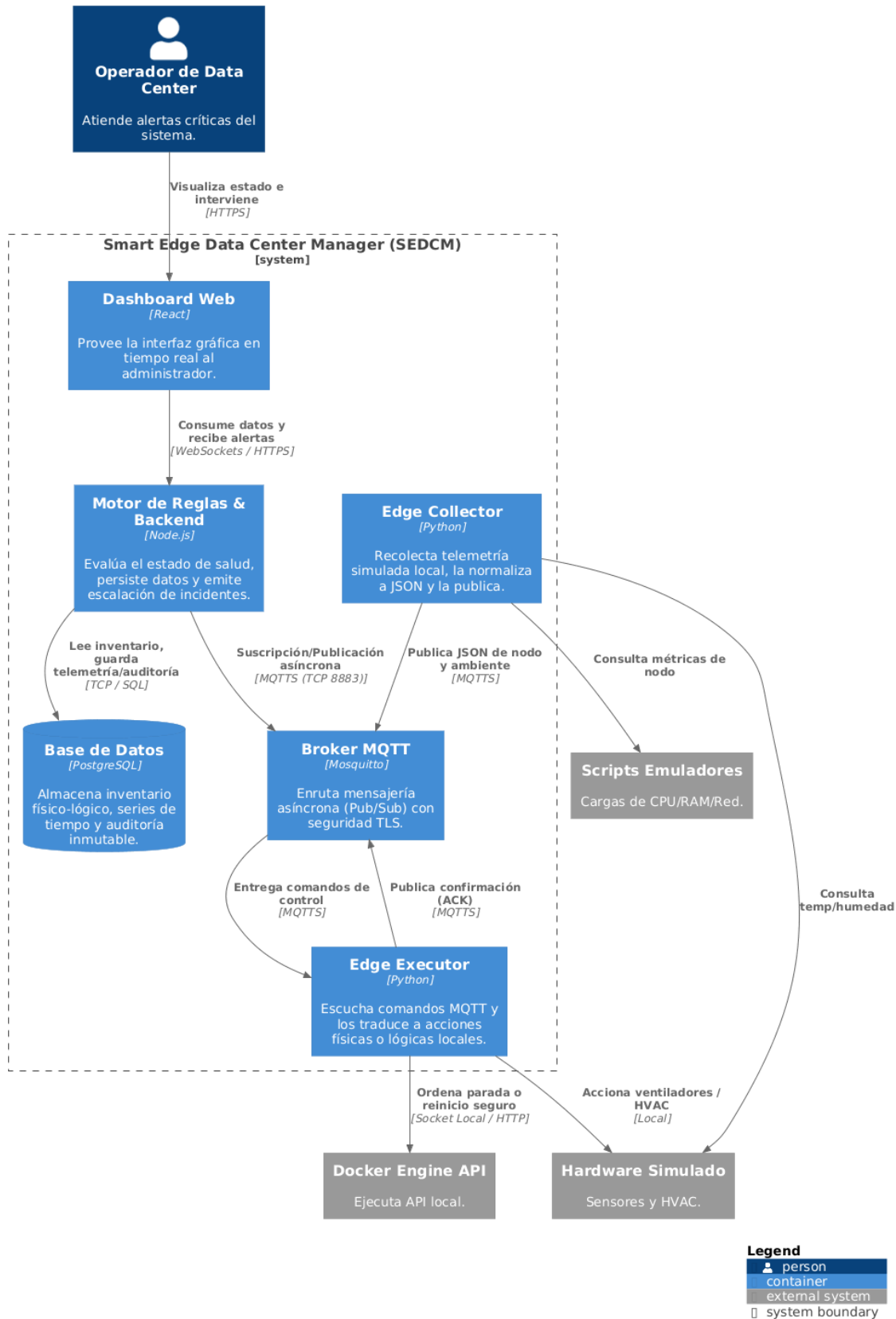
11.2.2 C2 Diagrama de Contenedores

El diagrama de contenedores (Nivel 2) expone la arquitectura interna del sistema SEDCM, detallando las unidades ejecutables y desplegables que lo conforman, así como las tecnologías seleccionadas para la comunicación en red. Este nivel ilustra claramente la topología distribuida (LAN On-Premise) y el paradigma de desacoplamiento asíncrono.

Descripción de los Contenedores Internos (SEDCM):

- **Aplicación Web (Dashboard):** Interfaz gráfica de usuario servida en el navegador. Se conecta al Backend mediante WebSockets para proveer visualización de telemetría y alertas con latencia casi nula, permitiendo al operador intervenir manualmente si es necesario.
- **Motor de Reglas (Backend API):** Aplicación central (Python) que actúa como el cerebro del sistema. Consume constantemente la telemetría del Broker MQTT, evalúa los datos, calcula el "Estado de Salud", persiste la información histórica y emite comandos de escalación automática.
- **Broker MQTT (Middleware Local):** Servidor de mensajería asíncrona configurado con MQTTS (TLS/SSL). Es el bus central de la red que enruta todos los mensajes bajo el patrón de Publicación/Suscripción, otorgando tolerancia a desconexiones temporales de los nodos de borde.
- **Base de Datos Relacional (PostgreSQL):** Repositorio central de persistencia. Almacena de forma normalizada el inventario de la infraestructura, las series de tiempo de la telemetría y la bitácora inmutable de auditoría de los comandos ejecutados.
- **Edge Collector (Agente de Borde):** Servicio ligero desplegado en los racks físicos (LAN). Su responsabilidad es extraer las métricas de los scripts emuladores internos y de los simuladores ambientales, transformar los datos a formato JSON y publicarlos en la red.
- **Edge Executor (Actuador de Borde):** Servicio residente en los racks que mantiene una suscripción activa a su tópico de control. Al recibir un comando de mitigación, interactúa directamente con la API local de Docker o los simuladores físicos para ejecutar la acción solicitada (ej. `soft_reboot` o `hard_shutdown`).

Diagrama de Contenedores (Nivel 2) - SEDCM

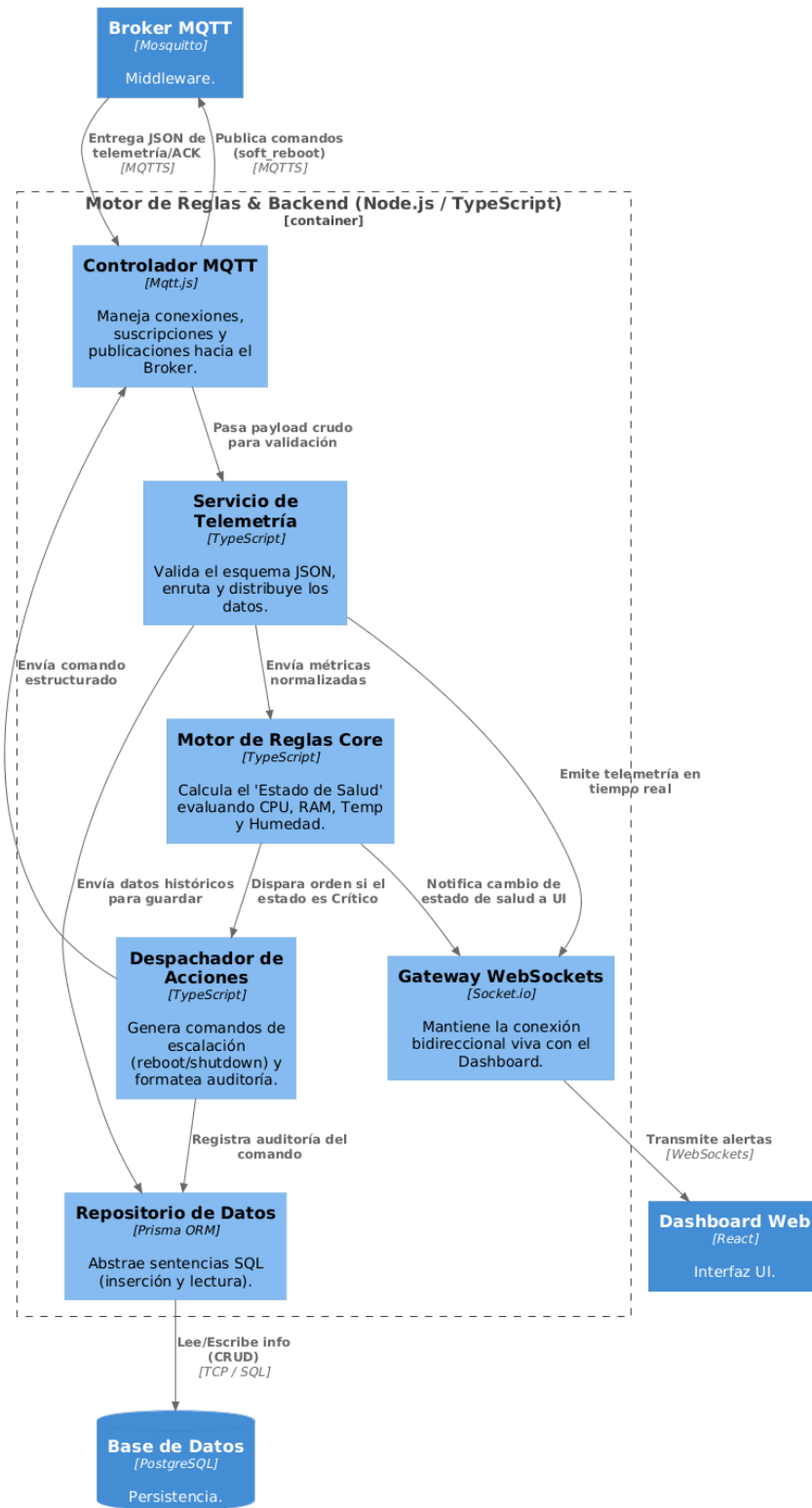


11.2.3 C3 - Diagrama de Componentes

Descripción de los Componentes Internos:

- **Controlador MQTT (MQTT Handler):** Componente (basado en mqtt.js) encargado exclusivamente de la capa de red. Maneja la conexión con el Broker Mosquitto, se suscribe a los tópicos de telemetría y publica los comandos de mitigación.
- **Servicio de Telemetría (Telemetry Service):** Recibe el payload JSON crudo de la red, lo valida, lo normaliza y lo distribuye a los demás componentes del sistema.
- **Motor de Reglas Core (Rules Engine):** El núcleo lógico del sistema. Implementa los algoritmos de histéresis térmica y las políticas de escalación de incidentes (saturación emulada de CPU/RAM). Evalúa continuamente las métricas entrantes contra los umbrales definidos.
- **Despachador de Acciones (Action Dispatcher):** Se activa cuando el Motor de Reglas detecta una anomalía. Es responsable de formatear el comando estructurado (ej. soft_reboot) y generar el registro para la bitácora de auditoría.
- **Gateway WebSockets (WS Gateway):** Interfaz de comunicación bidireccional (basada en socket.io) que mantiene conexiones vivas con los clientes del Dashboard (React) para emitir telemetría y alertas con latencia casi nula.
- **Repositorio de Datos (Data Repository):** Capa de abstracción de datos utilizando Prisma ORM. Encapsula todas las consultas SQL hacia PostgreSQL (inserción de métricas, actualización de confirmaciones ACK y lectura de inventario).

Diagrama de Componentes (Nivel 3) - Backend (Node.js) SEDCM



Legend

- container
- component
- container boundary

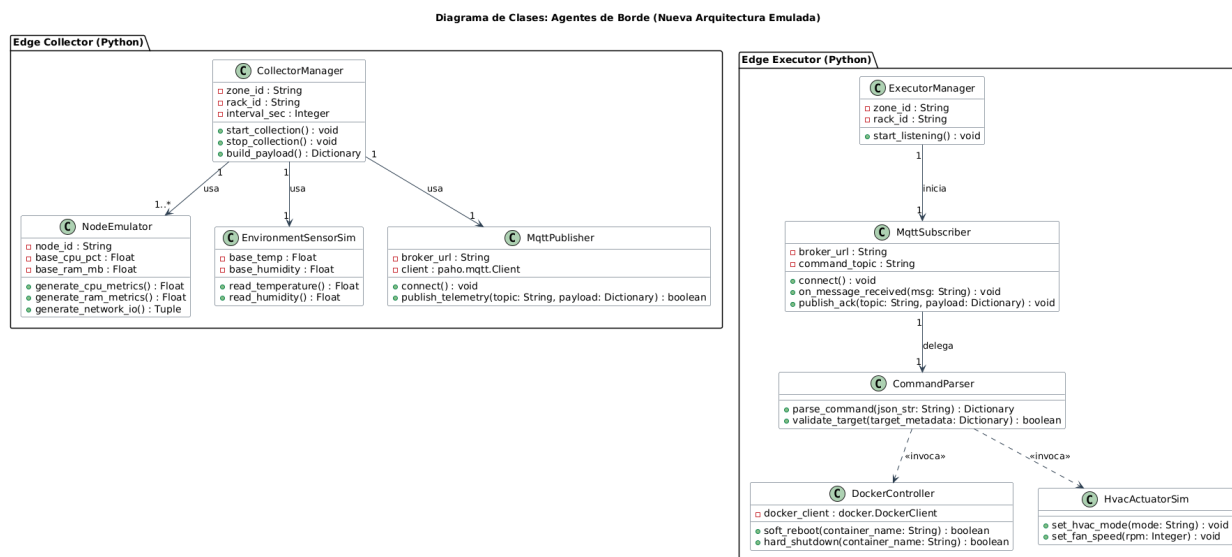
11.2.4 C4

11.3 Diagrama de Clases de Diseño (Agentes de Borde y Emuladores)

El siguiente diagrama de clases expone el diseño Orientado a Objetos (POO) de los componentes de software que residirán en los equipos físicos de borde (Zonas). Estos scripts, desarrollados en Python, son responsables de la emulación de cargas de trabajo, la recolección de telemetría y la ejecución de la escalación de incidentes local.

Descripción de las clases principales:

- **NodeEmulator (Emulador de Carga):** Representa el script interno de los contenedores Docker. Genera fluctuaciones controladas de CPU y RAM para simular estados de salud (Normal, Warning, Crítico) durante la demostración.
- **EnvironmentSimulator (Simulador Ambiental):** Mantiene el estado físico virtual del rack. Calcula constantemente la temperatura y la humedad, y reacciona a los cambios de estado del actuador HVAC.
- **EdgeCollector (Agente de Ingesta):** Clase orquestadora que se ejecuta en bucle (run_loop). Instancia conexiones hacia los emuladores, extrae sus datos brutos, los formatea en los esquemas JSON normalizados y los publica en el Broker MQTT.
- **EdgeExecutor (Agente de Acción):** Mantiene una suscripción activa al tópico de control del rack. Al recibir un *payload* del Backend, invoca los métodos de mitigación utilizando la API de Docker (soft_reboot, hard_shutdown) o interactuando con el simulador ambiental (set_hvac_mode), y finalmente publica un acuse de recibo (ACK).



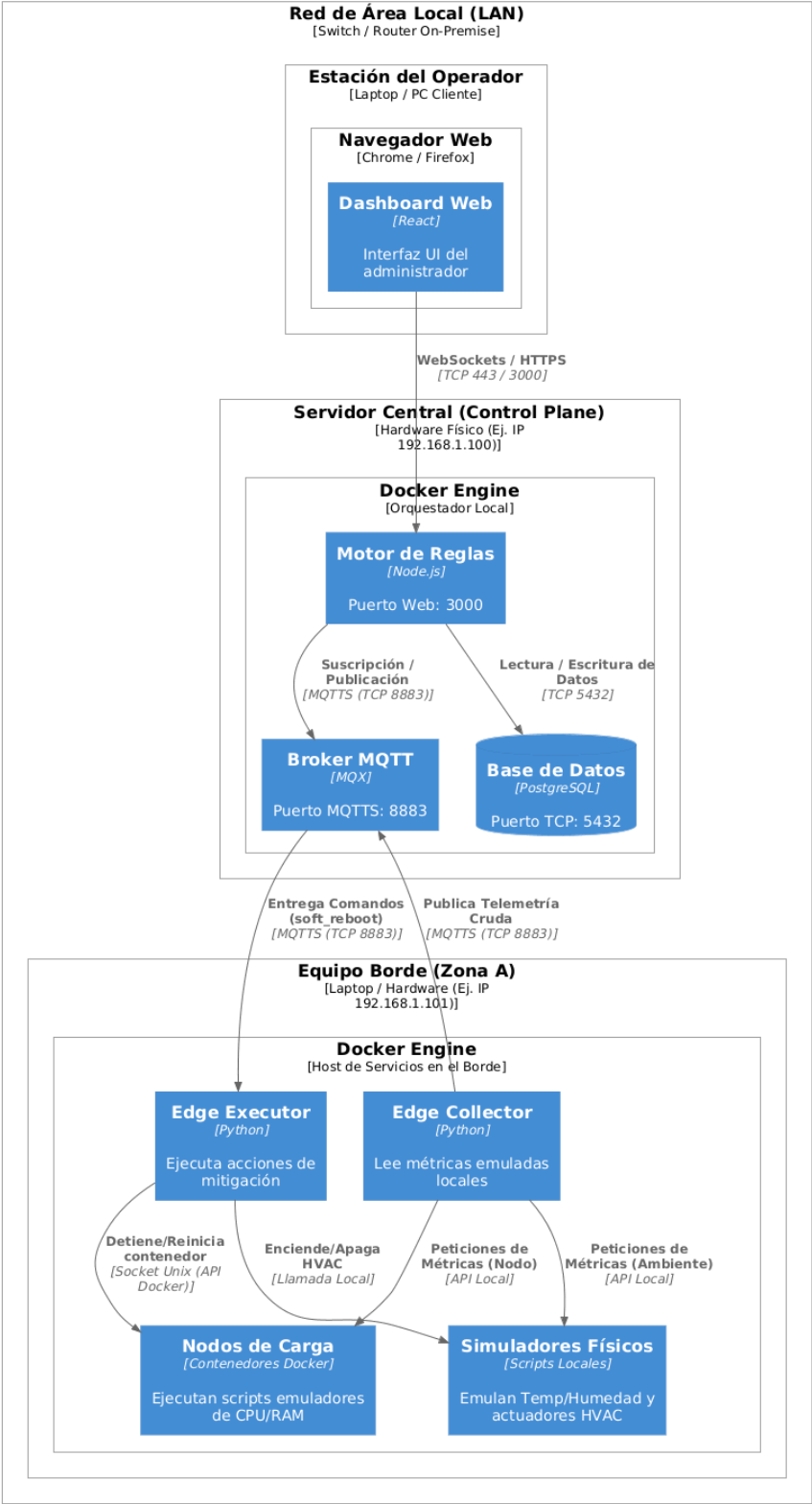
11.4 Diagrama de Despliegue (Topología Física LAN)

El diagrama de despliegue ilustra la topología física y la asignación de los contenedores de software al hardware subyacente. Para la demostración del Producto Mínimo Viable (MVP), el sistema SEDCM se implementará como un sistema distribuido que opera sobre una Red de Área Local (LAN). El entorno estará segmentado en múltiples equipos computacionales conectados al mismo conmutador (Switch) o punto de acceso inalámbrico, garantizando el aislamiento de red y la comunicación segura.

Nodos Físicos Principales:

1. **Servidor Central (Control Plane):** Un equipo físico designado (ej. IP 192.168.1.100) que centraliza la lógica pesada. Aloja el entorno Docker principal donde se ejecutan el Motor de Reglas (Backend Node.js), la base de datos relacional PostgreSQL (Puerto 5432) y el Broker MQTT Mosquitto asegurado con TLS/SSL (Puerto 8883).
2. **Equipos de Borde (Edge Nodes - Zonas):** Computadoras (Laptops adicionales) que representan lógicamente las Zonas y administran sus respectivos Racks. En cada equipo se ejecuta un demonio de Docker que aloja las cargas de trabajo emuladas (Nodos de Carga), los scripts de simulación ambiental (Temperatura/Humedad) y los agentes en Python (*Edge Collector* y *Edge Executor*).
3. **Estación del Operador:** Cualquier computadora conectada a la misma subred local. A través de un navegador web, accede a la dirección IP del servidor central para renderizar el Dashboard (React) y establecer la conexión en tiempo real vía WebSockets.

Diagrama de Despliegue (Nivel Físico) - SEDCM (Demostración LAN modular)



Legend
container
node

12. Modelo de Interfaz Gráfica de Usuario (

El Dashboard web, desarrollado en React, actúa como la única interfaz humana hacia el sistema distribuido. Su diseño prioriza la "Conciencia Situacional" (*Situational Awareness*), permitiendo al operador identificar instantáneamente el estado de salud de la red y ejecutar acciones de mitigación manuales cuando las reglas de negocio lo permitan.

12.1 Distribución Espacial (Layout)

La interfaz gráfica se dividirá en tres paneles principales para evitar la saturación cognitiva del usuario durante un evento de emergencia:

1. **Panel de Inventario Dinámico (Izquierda):** Un árbol de navegación jerárquico que refleja las tablas lógicas (zonas, racks y nodos). El color de cada elemento cambiará en tiempo real (Verde = Normal, Amarillo = Warning, Rojo = Crítico) según el *Estado de Salud* calculado por el Motor de Reglas.
2. **Panel de Telemetría y Control (Centro):** Muestra los datos específicos del activo seleccionado en el árbol.
 - **Gráficos de Series de Tiempo:** Renderizan el histórico a corto plazo del consumo emulado de CPU/RAM, Temperatura y Humedad. Incluyen líneas horizontales punteadas que marcan los umbrales de alerta definidos en las reglas de operación.
 - **Panel de Mitigación Manual:** Botones de acción (soft_reboot, hard_shutdown, set_hvac_mode). El sistema de seguridad bloqueará (deshabilitará) el botón de reinicio si las variables físicas no se encuentran en rangos de recuperación segura (ej. Temperatura > 42°C).
3. **Consola de Auditoría en Vivo (Derecha):** Un *feed* de texto tipo terminal, conectado por WebSockets, que muestra un flujo ininterrumpido de alarmas, comandos emitidos por el sistema autónomo y los acuses de recibo (ACKs) provenientes del *Edge Executor*

12.2 Wireframe Conceptual

(Nota para el equipo de desarrollo: El siguiente esquema dictamina la estructura mínima indispensable que debe programarse en la capa de presentación).

13. Arquitectura de Tópicos (Namespace MQTT para EMQX)

El sistema SEDCM utiliza un esquema de enrutamiento jerárquico a través del broker EMQX. Este diseño garantiza el desacoplamiento total de la red y permite el uso eficiente de comodines (*wildcards* + y #), lo que facilita que el Backend central escuche a todo el Data Center con una sola conexión, mientras que los Nodos de Borde solo escuchan lo que les corresponde físicamente.

A continuación, se detalla la matriz de comunicación de la red LAN:

13.1 Tópicos de Telemetría (Flujo: Borde → Servidor Central)

Estos tópicos son de alta frecuencia. Los Nodos de Borde publican sus datos aquí, y el Motor de Reglas (Backend) es el único suscriptor.

Tópico MQTT	Publicador	Suscriptor	Propósito / Frecuencia
dc/telemetria/zona/{Z}/rack/{R}/nodo/{N}	Edge Collector	Backend	Propósito: Envía el JSON con el consumo de CPU, RAM e I/O de red del emulador.Frecuencia: Cada 5 segundos.
dc/telemetria/zona/{Z}/rack/{R}/ambiente	Edge Collector	Backend	Propósito: Envía el JSON con la Temperatura y Humedad del simulador físico.Frecuencia: Cada 10 segundos.

13.2 Tópicos de Control y Reacción (Flujo: Servidor Central → Borde)

Estos tópicos son de baja frecuencia (eventos). El Backend emite comandos aquí solo cuando se rompe una regla de negocio. Los *Edge Executors* están suscritos a su ruta específica.

Tópico MQTT	Publicador	Suscriptor	Propósito / Frecuencia
dc/control/zona/{Z}/rack/{R}	Backend	Edge Executor	Propósito: Recibir comandos de escalación estructurados (soft_reboot, hard_shutdown, set_hvac_mode). El Executor de un rack no se entera de lo que pasa en otro rack.Frecuencia: Solo por evento de anomalía.

dc/ack/zona/{Z}/rack/{R}	Edge Executor	Backend	Propósito: Retornar la confirmación (Acuse de Recibo) de que una acción de control fue ejecutada exitosamente en el hardware/Docker para cerrar la bitácora de auditoría.Frecuencia: Inmediatamente después de procesar un control.
--------------------------	------------------	---------	---

13.3 Tópicos de Estado y Resiliencia (LWT - Last Will and Testament)

Para cumplir con la "Regla de Resiliencia del Agente", el sistema utiliza la función LWT nativa de EMQX. Si una laptop (Zona) se queda sin Wi-Fi, EMQX detecta la desconexión del socket TCP y publica automáticamente un mensaje de "muerte" en nombre del agente caído.

Tópico MQTT	Publicador	Suscriptor	Propósito / Frecuencia
dc/estado/zona/{Z}/agente/{A}	EMQX (LWT) / Agente Python	Backend	Propósito: Anunciar si el Edge Collector o Executor está ONLINE o OFFLINE.Payload: {"status": "offline", "timestamp": "..."}Frecuencia: Al conectar y desconectar de la red.

13.4 Estrategia de Suscripción (Wildcards)

Para optimizar el uso de memoria en el servidor central de Node.js, el Backend no se suscribirá a cada nodo individualmente. En su lugar, utilizará comodines de nivel único (+) y nivel múltiple (#) al iniciar la conexión con EMQX:

- **Suscripción del Backend para toda la telemetría:** El Backend se suscribe a dc/telemetria/#. Con esto, un solo *listener* en Node.js captura la temperatura de todos los racks y el CPU de todos los nodos de manera centralizada.

- **Suscripción del Edge Executor (Ej. Rack A1):** El script en Python del rack A1 se suscribirá exclusivamente a dc/control/zona/A/rack/A1. Ignorará por completo el tráfico de la Zona B o del Rack A2, garantizando aislamiento físico y lógico.